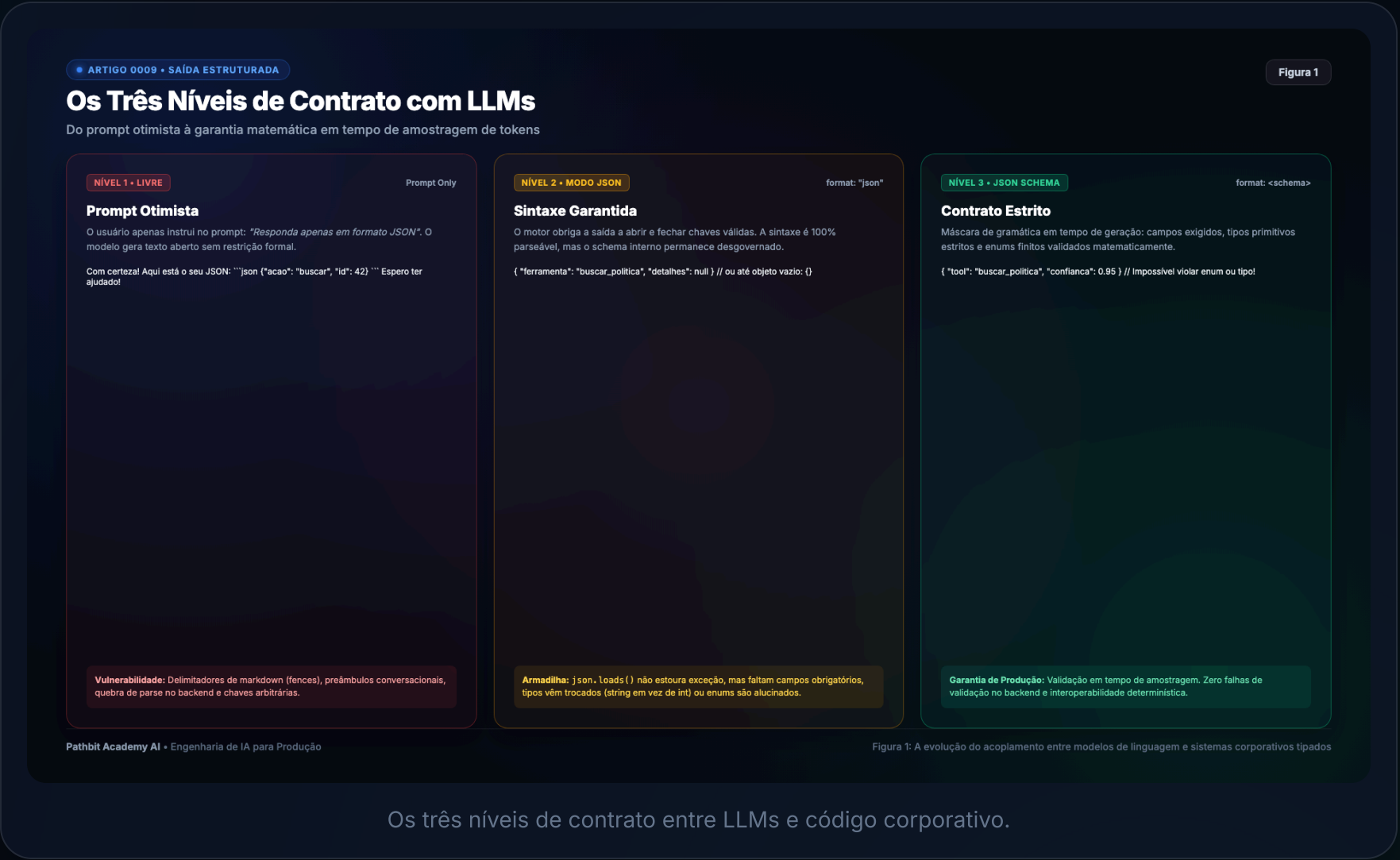


Saída Estruturada com LLMs: Do Prompt ao Schema Validado

Este módulo prova, com dados medidos em modelos locais, como sair do prompt ingênuo e garantir 100% de conformidade de schema sem quebra de parse no backend.

- Leitura completa no artigo + notebook + CSVs no repositório

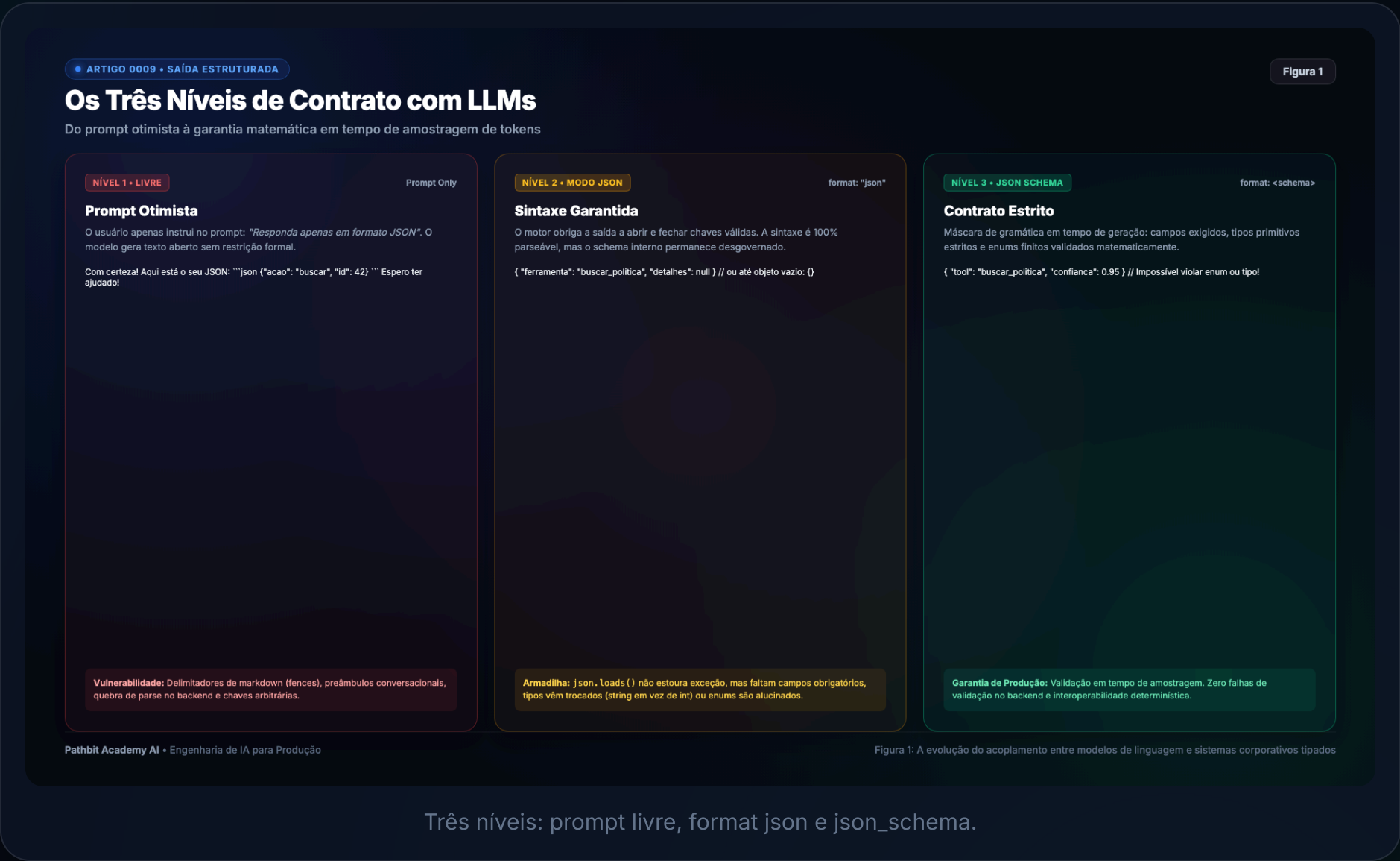


O PROBLEMA DA FRAGILIDADE

Prompt otimista não é contrato de engenharia

Pedir JSON no texto livre resulta em markdown fences, preâmbulos e chaves alucinadas. O `json.loads` quebra e derruba o pipeline de produção.

Em produção corporativa, confiar em prompt livre para gerar JSON é assumir débito técnico no primeiro deploy.



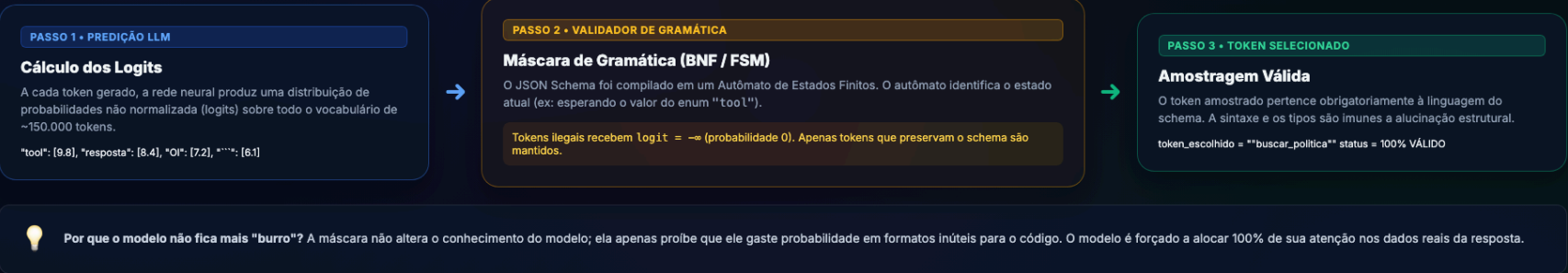
Grammar-Guided Sampling: máscara de logits em tempo real

O schema é compilado em uma máquina de estados finitos. A cada token gerado pelo modelo, tokens inválidos recebem probabilidade zero.

O modelo não tem como errar a sintaxe: a máscara física impede qualquer token ilegal.

Como Funciona a Decodificação Constrangida

Grammar-Guided Sampling: máscara de logits via Autômato de Estados Finitos (FSM)



Logits masking guiado por Autômato de Estados Finitos.

EVIDÊNCIA EMPÍRICA

Números medidos: de 55% a 100% de conformidade

No modo livre, quase 30% das chamadas sequer parseiam. Com `format: <schema>`, a conformidade salta para 100% e a assertividade de negócio atinge o teto.

Schema estrito não engessa o modelo: ele concentra a probabilidade na semântica correta.



O CUSTO DO RETRY

O mito do retry compensatório

Tentar consertar saída livre com novo prompt custa o dobro da latência. A imposição de schema adiciona overhead desprezível e resolve na 1ª tentativa.

Retries devem tratar regras de negócio semânticas, nunca sintaxe quebrada.



Contrato desacoplado e tipagem estrita

O contrato em Pydantic/JSON Schema vira especificação única. O backend deserializa com garantia total e audita cada decisão em CSV/JSONL.

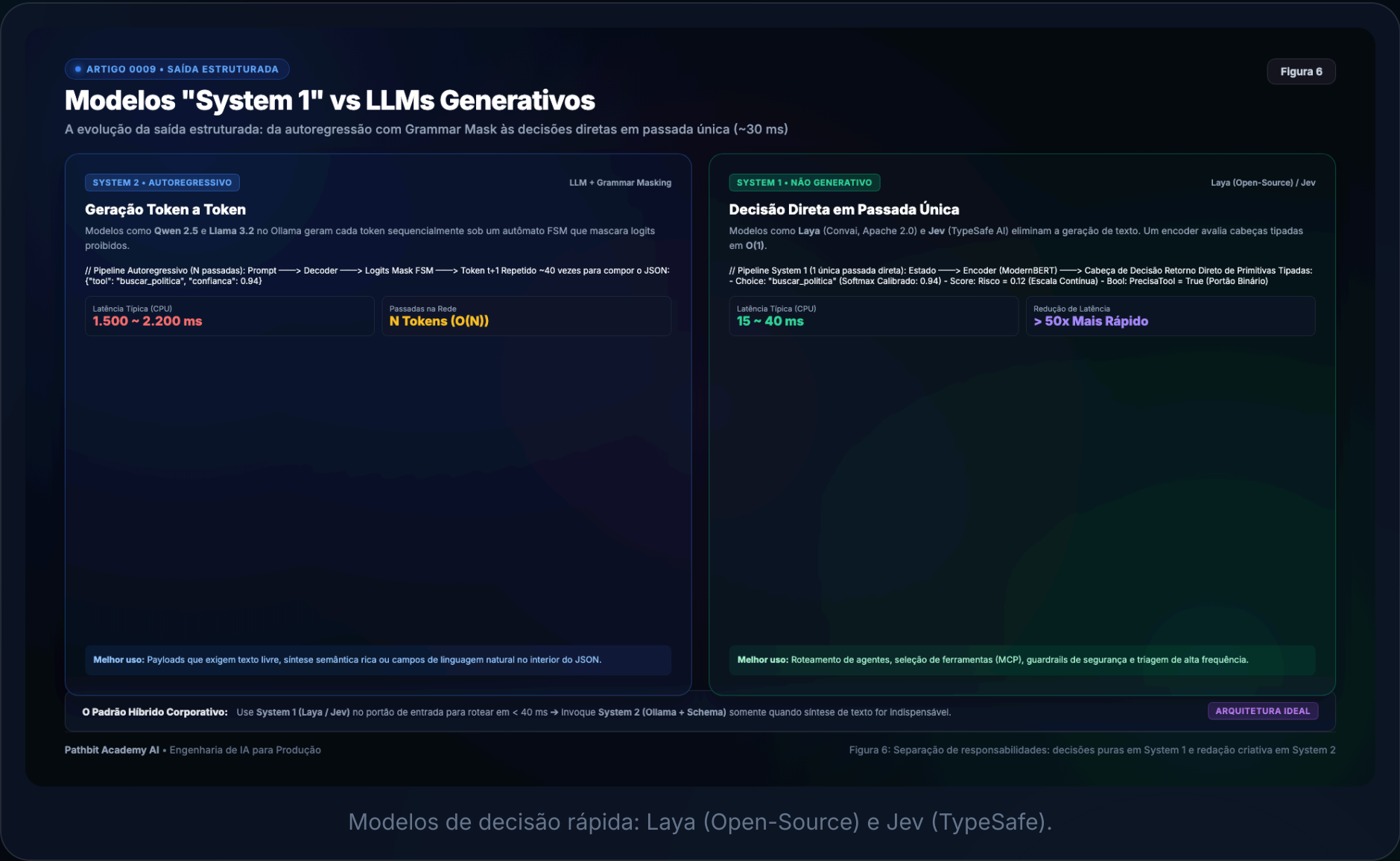
A saída estruturada transforma um gerador probabilístico de texto em uma API determinística.



Modelos System 1: Decisão Direta em ~30 ms

Por que gerar 40 tokens sequenciais se o sistema precisa apenas de uma decisão tipada? Modelos System 1 usam passada única (O(1)) para Choice, Score e Bool com mais de 50x de redução de latência.

A separação moderna: decisões em System 1 (< 40ms) e redação rica em System 2.



Saída Estruturada + Modelos System 1

Do prompt otimista à garantia formal via Grammar-Guided Sampling, com benchmarks empíricos em modelos locais.

- 100% de conformidade de schema sem quebra de parse
- Desacoplamento com Pydantic e JSON Schema estrito
- Avaliação empírica de decisores System 1 vs autoregressão

github.com/pathbit/pathbit-academy-ai

REPOSITÓRIO OFICIAL

github.com/pathbit/pathbit-academy-ai

Código 100% aberto, sem chave de API e pronto para execução local.

Acessar Laboratório Completo →

Como Funciona a Decodificação Constrangida

Grammar-Guided Sampling: máscara de logits via Autômato de Estados Finitos (FSM)

PASSO 1 - PREDIÇÃO LLM

Cálculo dos Logits

A cada token gerado, o modelo neural produz uma distribuição de probabilidades não normalizada (logits) sobre todo o vocabulário de ~100.000 tokens.

"tool": [0.8], "response": [0.4], "top": [0.2], "...": [0.6]

PASSO 2 - VALIDADOR DE GRAMÁTICA

Máscara de Gramática (BNF / FSM)

O JSON Schema foi compilado em um Autômato de Estados Finitos. O autômato identifica o estado atual (ex: esperando o valor do enum "tool").

Tokens legais recebem `logit + 1` → (probabilidade 0). Apenas tokens que preservam o schema são mantidos.

PASSO 3 - TOKEN SELECIONADO

Amostragem Válida

O token amostrado pertence obrigatoriamente à linguagem do schema. A sintaxe e os tipos são imunes à elisãoção estrutural.

token_selecionado = "buscar_politica" status = 100% VÁLIDO

Por que o modelo não fica mais "burro"? A máscara não altera o conhecimento do modelo; ela apenas proíbe que ele gaste probabilidade em formatos inúteis para o código. O modelo é forçado a alocar 100% de sua atenção nos dados reais da resposta.

Conformidade e Assertividade por Modo de Execução

54 execuções controladas com qwen2.5 (0.5B e 1.5B) e llama3.2 (1B) no Ollama

MODO LIVRE

72.2% parse_ok

Quase 30% das saídas falham no json.loads direto sem limpeza manual.

Instável

FORMAT: "JSON"

100% parse_ok

Sintaxe sempre válida, mas tipos e enums internos ainda falham.

Sintático

FORMAT: "SCHEMA"

100% schema_ok

Zero falhas de contrato, 100% dos campos, tipos e enums respeitados.

Garantido

Conformidade com Schema

55.6%

Acerto Semântico

44.4%

Conformidade com Schema

66.7%

Acerto Semântico

55.6%

Conformidade com Schema

100.0%

Acerto Semântico

88.9%

Modelos "System 1" vs LLMs Generativos

A evolução da saída estruturada: da autoregressão com Grammar Mask às decisões diretas em passada única (~30 ms)

SYSTEM 2 - AUTOREGRESSIVO

LLM + Grammar Masking

Geração Token a Token

Modelos como Qwen 2.5 e Llama 3.2 no Ollama geram cada token sequencialmente sob um autômato FSM que mascara logits proibidos.

Pipeline Autoregressivo (N passadas): Prompt →> Decider →> Logits Mask FSM →> Token (v1 Repetido ~40 vezes para compor o JSON: {"tool": "buscar_politica", "confianca": 0.94})

Latência Típica (CPU)

1.500 ~ 2.200 ms

Passadas na Rede

N Tokens (O(N))

Melhor uso: Playbooks que exigem texto livre, síntese semântica rica ou campos de linguagem natural no interior do JSON.

O Padrão Híbrido Corporativo: Usa System 1 (Laysa / Jey) no ponto de entrada para rotar em < 40 ms → invoca System 2 (Ollama + Schema) somente quando síntese de texto for indispensável.

SYSTEM 1 - NÃO GENERATIVO

Laysa (Open-Sourced) / Jey

Decisão Direta em Passada Única

Modelos como Laysa (Convai), Apache 2.0 e Jey (TypeSafe AI) eliminam a geração de texto. Um encoder avalia cabeças tipadas em O(1).

Pipeline System 1 (1 única passada direta): Estado →> Encoder (ModernaBERT) →> Cabeça de Decisão Retorna Direto da Primeira Tipagem: - Choice: "buscar_politica" (Software Calibrado: 0.94) - Score: Risco = 0.02 (Escala Contínua) - Root: Preprocess = True (Ponto de Entrada)

Latência Típica (CPU)

15 ~ 40 ms

Redução de Latência

> 50x Mais Rápido

Melhor uso: Roteamento de agentes, seleção de ferramentas (MCP), guardrails de segurança e triagem de alta frequência.

ARQUITETURA IDEAL

8 / 8

github.com/pathbit/pathbit-academy-ai

0009_saida_estruturada